

Documentación técnica

Portafolio y web de servicios IT

Guía completa del repositorio `redwan-alexandru-teodor.github.io`

para entender, ejecutar y modificar cualquier parte del proyecto

Autor: Redwan Alexandru Teodor

Técnico en Sistemas y Redes – Sevilla, España

Octubre de 2026

Sitio en producción: <https://redwan-alexandru-teodor.github.io/>

Perfil profesional: <https://www.linkedin.com/in/redwan-alexandru-teodor/>

Índice

1. Introducción	4
1.1. Qué ofrece el sitio	4
2. Arquitectura general	4
2.1. Exportación estática	4
2.2. Flujo de datos de configuración	4
2.3. Principios de diseño	5
3. Stack tecnológico	5
3.1. Conceptos clave para quien no conoce el stack	5
4. Puesta en marcha en local	6
4.1. Requisitos	6
4.2. Pasos	6
4.3. Variables de entorno	6
5. Estructura del repositorio	6
6. Configuración del sitio	7
6.1. <code>lib/site-config.ts</code>	7
6.2. <code>next.config.mjs</code>	8
6.3. <code>package.json</code> y <code>pnpm</code>	8
7. Contenido editable: datos	8
7.1. Servicios (<code>data/servicios.ts</code>)	8
7.2. Tecnologías (<code>tecnologias</code>)	9
7.3. Proyectos (<code>data/proyectos/</code>)	9
7.3.1. Markdown admitido en <code>readme</code>	9
8. Componentes	10
8.1. Página principal (<code>app/page.tsx</code>)	10
8.2. Cabecera (<code>site-header.tsx</code>)	10
8.3. Hero (<code>hero.tsx</code>)	10
8.4. Sobre mí (<code>about.tsx</code>)	10
8.5. Servicios (<code>services.tsx</code>)	10
8.6. Proceso (<code>process.tsx</code>)	10
8.7. Proyectos (<code>proyectos.tsx</code>)	10

8.8. Contacto (<code>contact.tsx</code>)	11
8.9. Pie de página (<code>site-footer.tsx</code>)	11
8.10. Utilidades de interfaz	11
8.11. Hooks y utilidades (<code>lib/</code>)	11
9. Formulario de contacto	11
9.1. Flujo completo	11
9.2. Validaciones	12
9.3. Identificador único (<code>lib/mail.ts</code>)	12
9.4. Países y teléfonos (<code>lib/paises.ts</code>)	12
9.5. Cómo cambiar el destino del correo	12
9.6. Si quieres un envío real sin cliente de correo	13
10. Estilos y tema	13
10.1. Tokens de color (<code>app/globals.css</code>)	13
10.2. Otros estilos globales	13
10.3. Fuente	13
10.4. Modo claro	14
11. SEO y metadatos	14
11.1. <code>app/layout.tsx</code>	14
11.2. Imagen para redes (<code>public/og_image.png</code>)	14
11.3. <code>sitemap.ts</code> y <code>robots.ts</code>	14
11.4. Icono	14
12. Despliegue en GitHub Pages	14
12.1. Flujo automático	14
12.2. Configuración inicial del repositorio	15
12.3. Nombre del repositorio y <code>basePath</code>	15
12.4. El archivo <code>.nojekyll</code>	15
12.5. Caché de compilación	15
13. Recetas: cómo modificar...	15
13.1. Cambiar mi nombre o cargo	15
13.2. Cambiar la foto de perfil	15
13.3. Cambiar la ciudad	15
13.4. Cambiar textos de una sección	16
13.5. Cambiar el color principal	16

13.6. Ocultar la insignia de disponibilidad	16
13.7. Reordenar o eliminar una sección	16
13.8. Añadir una sección nueva	16
13.9. Sustituir el PDF de documentación	16
13.10Cambiar el texto legal o las condiciones	16
13.11Añadir un país o cambiar el país por defecto	16
13.12Cambiar el prefijo del asunto del correo	16
13.13Cambiar el tiempo de respuesta	17
13.14Reutilizar este sitio como plantilla	17
14.Accesibilidad	17
15.Resolución de problemas	17
16.Buenas prácticas al contribuir	18
17.Autor y licencia	18

1 Introducción

Este documento describe el repositorio del sitio web personal y profesional de Redwan Alexandru Teodor. El objetivo es que cualquier persona, aunque no haya participado en su desarrollo, pueda:

- entender qué hace el sitio y cómo está construido;
- ejecutarlo en su ordenador;
- modificar textos, imágenes, colores, servicios, proyectos y formulario;
- publicarlo de nuevo en GitHub Pages sin romper nada.

El sitio es una **página estática**: no hay servidor propio, base de datos ni API. Todo el contenido se genera durante la compilación (*build*) y se publica como archivos HTML, CSS y JavaScript en GitHub Pages.

1.1 Qué ofrece el sitio

- Presentación profesional (*hero*), sección *Sobre mí*, catálogo de servicios, proceso de trabajo, galería de proyectos y formulario de contacto.
- Servicios técnicos ofrecidos en régimen de colaboración gratuita a cambio de autorizar su mención en el portafolio.
- Formulario que no envía datos a ningún servidor: prepara un correo (<mailto:>) en el cliente del visitante.
- Aviso legal (RGPD y LSSI-CE) y visor de un PDF de documentación.
- SEO completo: metadatos, Open Graph, JSON-LD, `sitemap.xml` y `robots.txt`.

2 Arquitectura general

2.1 Exportación estática

El proyecto usa Next.js con la opción `output: 'export'`. Al ejecutar `pnpm build` se genera la carpeta `out/` con el sitio completo. Esa carpeta es lo único que se publica.

```
Codigo fuente (app/, components/, data/, lib/, public/)
|
|  pnpm build  (Next.js, export estatico)
v
carpeta out/  -->  GitHub Pages
```

2.2 Flujo de datos de configuración

```
deploy.yml --(NEXT_PUBLIC_BASE_PATH, NEXT_PUBLIC_CONTACT_EMAIL)--> lib/site-config.ts
site-config --> layout.tsx, sitemap.ts, robots.ts, todas las llamadas a asset()
data/servicios.ts --> components/services.tsx y layout.tsx (JSON-LD)
data/proyectos    --> components/proyectos.tsx y sitemap.ts (imagenes)
lib/mail.ts        --> mail-modal.tsx y contact-form.tsx
lib/use-scroll-lock.ts --> mail-modal, contact-form, site-footer, proyectos
```

2.3 Principios de diseño

- **Una sola fuente de verdad:** los servicios viven en `data/servicios.ts`; la configuración del despliegue, en `lib/site-config.ts`.
- **Sin dependencias innecesarias:** el Markdown de los proyectos se renderiza con una función propia, sin librerías externas.
- **Correo fuera del código:** la dirección de contacto no está escrita en el repositorio público.
- **Accesible y sin JavaScript obligatorio para ver el contenido:** las animaciones nunca ocultan contenido si JavaScript falla.

3 Stack tecnológico

Tecnología	Versión	Para qué se usa	Dónde
Node.js	24 (LTS)	Entorno de compilación	<code>deploy.yml</code>
pnpm	según <code>packageManager</code>	Gestor de paquetes	<code>package.json</code>
Next.js	16 (App Router)	Framework y exportación estática	<code>next.config.mjs</code> , <code>app/</code>
React	19	Interfaz y estado de componentes	<code>components/</code>
TypeScript	5.7	Tipado; los errores rompen el build	<code>tsconfig.json</code>
Tailwind CSS	4	Estilos y tokens de color OKLCH	<code>app/globals.css</code>
shadcn/ui y Base UI	4 / 1.5	Primitivo de botón	<code>components/ui/</code>
Lucide React	1.x	Iconos SVG	<code>components/*</code>
tw-animate-css	1.4	Animaciones de modales	<code>app/globals.css</code>
clsx, tailwind-merge, cva	–	Composición de clases	<code>lib/utils.ts</code>
Schema.org JSON-LD	–	Datos estructurados para buscadores	<code>app/layout.tsx</code>
GitHub Pages y Actions	–	Alojamiento y CI/CD	<code>.github/workflows/</code>

3.1 Conceptos clave para quien no conoce el stack

Next.js Framework sobre React. En este proyecto se usa solo para generar HTML estático, enrutado y metadatos.

App Router Sistema de rutas basado en la carpeta `app/`. Cada `page.tsx` es una página; `layout.tsx` envuelve a todas.

Componente de cliente Archivo que empieza con `'use client'`. Puede usar estado y eventos del navegador. Los que no lo llevan se ejecutan solo durante el build.

Tailwind CSS Se estiliza con clases en el propio HTML (por ejemplo `px-4 text-sm`) en lugar de hojas de estilo separadas.

Alias @/ Equivale a la raíz del proyecto: @/lib/utils es ./lib/utils.

4 Puesta en marcha en local

4.1 Requisitos

- Node.js 24 (o una versión LTS reciente).
- pnpm (puede activarse con `corepack enable`).
- Git.

4.2 Pasos

```
# 1. Clonar el repositorio
git clone https://github.com/Redwan-Alexandru-Teodor/redwan-alexandru-teodor.github.io.git
cd redwan-alexandru-teodor.github.io

# 2. Instalar dependencias
pnpm install

# 3. Definir el correo de contacto para pruebas locales
echo "NEXT_PUBLIC_CONTACT_EMAIL=tu@correo.com" > .env.local

# 4. Servidor de desarrollo con recarga automática
pnpm dev
# -> abrir http://localhost:3000

# 5. Compilar la versión de producción
pnpm build
# -> el sitio completo queda en la carpeta out/
```

Nota: `pnpm start` no sirve un sitio exportado de forma estática. Para probar `out/` usa cualquier servidor estático, por ejemplo `npx serve out`.

4.3 Variables de entorno

Variable	Descripción
NEXT_PUBLIC_CONTACT_EMAIL	Correo al que se dirige el formulario. Si está vacía, el formulario se desactiva y muestra un aviso.
NEXT_PUBLIC_BASE_PATH	Prefijo de ruta (por ejemplo /inicio). Vacío si el repositorio se llama <usuario>.github.io. Lo rellena el workflow.
NEXT_PUBLIC_SITE_ORIGIN	Dominio de publicación. Por defecto <code>https://redwan-alexandru-teodor.github.io</code> .

El prefijo `NEXT_PUBLIC_` es obligatorio: indica a Next.js que la variable debe incluirse en el código que llega al navegador.

5 Estructura del repositorio

```

./
|-- .github/workflows/deploy.yml    Build y publicacion en GitHub Pages
|-- app/
|   |-- layout.tsx                 Layout raiz: fuente, metadatos, JSON-LD
|   |-- page.tsx                   Ensambla las secciones de la pagina
|   |-- globals.css                Tokens de color y estilos globales
|   |-- not-found.tsx              Pagina 404
|   |-- robots.ts                  Genera robots.txt
|   |-- sitemap.ts                  Genera sitemap.xml
|-- components/
|   |-- ui/button.tsx               Primitivo de boton (shadcn/Base UI)
|   |-- site-header.tsx             Menu fijo, seccion activa, CTA movil
|   |-- hero.tsx                    Presentacion inicial
|   |-- about.tsx                   Seccion "Sobre mi"
|   |-- services.tsx                Servicios y tabla de tecnologias
|   |-- process.tsx                 Proceso en 3 pasos
|   |-- proyectos.tsx               Galeria, visor con zoom, Markdown
|   |-- contact.tsx                 Seccion de contacto
|   |-- contact-form.tsx            Formulario completo
|   |-- mail-modal.tsx              Confirmacion de "Enviar correo"
|   |-- site-footer.tsx             Pie, aviso legal y visor PDF
|   |-- reveal.tsx                  Animacion de entrada al hacer scroll
|   |-- back-to-top.tsx             Boton de volver arriba
|-- data/
|   |-- servicios.ts                Servicios y tecnologias
|   |-- proyectos/
|       |-- index.ts                Lista ordenada de proyectos
|       |-- types.ts                Tipo ProyectoData
|       |-- pagina-web-servicios/index.ts
|-- lib/
|   |-- site-config.ts              basePath, siteUrl, correo, disponibilidad
|   |-- mail.ts                     ID unico, zona horaria, URL mailto
|   |-- paises.ts                   Prefijos telefonicos y validacion
|   |-- use-scroll-lock.ts           Bloqueo de scroll para modales
|   |-- use-active-section.ts        Seccion visible (resalta el menu)
|   |-- utils.ts                    Funcion cn()
|-- public/                          Archivos estaticos servidos tal cual
|   |-- images/perfil.jpg
|   |-- proyectos/<slug>/            Imagenes de cada proyecto
|   |-- .nojekyll, icon.svg, og_image.png, Documentacion.pdf
|-- components.json                  Configuracion de shadcn/ui
|-- next.config.mjs                  Configuracion de Next.js
|-- package.json, pnpm-workspace.yaml
|-- postcss.config.mjs, tsconfig.json
|-- README.md

```

Regla importante: los archivos binarios (imágenes, PDF) deben estar siempre en `public/`. La exportación estática solo copia esa carpeta al sitio final.

6 Configuración del sitio

6.1 lib/site-config.ts

Es el archivo central. Todo lo que varía según el despliegue o los datos personales está aquí.

```

export const basePath = process.env.NEXT_PUBLIC_BASE_PATH || ''
export const asset = (path: string) => `${basePath}${path}`
export const siteOrigin =
  process.env.NEXT_PUBLIC_SITE_ORIGIN || 'https://redwan-alexandru-teodor.github.io'

```

```
export const siteUrl = `${siteOrigin}${basePath}`
export const contactEmail = (process.env.NEXT_PUBLIC_CONTACT_EMAIL || '').trim()
export const disponible = true
```

`asset(ruta)` Antepone el `basePath` a una ruta de `public/`. Se debe usar siempre que se referencia una imagen o archivo propio: `asset('/images/perfil.jpg')`.

`siteUrl` URL base completa, usada en SEO, sitemap y JSON-LD.

`disponible` Si es `true`, el *hero* muestra el distintivo verde “Disponible para nuevos proyectos”. Ponlo a `false` para ocultarlo.

6.2 next.config.mjs

```
const basePath = process.env.NEXT_PUBLIC_BASE_PATH || ''
const nextConfig = {
  output: 'export',
  basePath,
  assetPrefix: basePath ? `${basePath}/` : undefined,
  trailingSlash: true,
  typescript: { ignoreBuildErrors: false },
  images: { unoptimized: true },
}
export default nextConfig
```

- `output: 'export'` activa la generación estática.
- `images.unoptimized` es necesario porque GitHub Pages no tiene servidor de optimización de imágenes.
- `ignoreBuildErrors: false` hace que un error de TypeScript falle el build y, por tanto, el despliegue.

6.3 package.json y pnpm

Los scripts disponibles son `dev`, `build` y `start`. El archivo `pnpm-workspace.yaml` contiene ajustes de pnpm (`minimumReleaseAgeExclude` para permitir versiones recientes de Next.js).

7 Contenido editable: datos

7.1 Servicios (data/servicios.ts)

Cada servicio es un objeto con esta forma:

```
interface Servicio {
  id: string // identificador; decide el icono
  label: string // titulo visible
  description: string // texto al desplegar la tarjeta
  tiempo: string // duracion orientativa
}
```

Para añadir un servicio:

1. Añade un objeto al array `servicios` en `data/servicios.ts`.
2. En `components/services.tsx`, añade su `id` al objeto `ICONOS` apuntando a un icono de Lucide (importado arriba). Si no lo haces, se usa el icono `Wrench` por defecto.
3. Si quieres que aparezca en el formulario, añade también su nombre al array `serviceOptions` de `components/contact-form.tsx`.

Los servicios también alimentan el JSON-LD de `layout.tsx`, por lo que el SEO se actualiza solo.

7.2 Tecnologías (tecnologias)

En el mismo archivo hay un array `tecnologias` con grupos `\{ area, items[] \}`. Se muestra como tabla “Entornos y herramientas con los que trabajo” al final de la sección de servicios.

7.3 Proyectos (data/proyectos/)

Cada proyecto es una carpeta con un `index.ts` que exporta un objeto `ProyectoData`:

```
export interface ProyectoData {
  id: string
  titulo: string
  subtítulo: string
  portada: string // ruta bajo public/, p. ej. /proyectos/slug/Portada.png
  galeria: string[] // imagenes adicionales, en orden
  readme: string // Markdown; cada "## Título" crea una pestaña
  github?: string // enlace opcional al repositorio
  tecnologias: string[] // etiquetas
}
```

Para añadir un proyecto nuevo:

1. Crea `data/proyectos/<slug>/index.ts` copiando el existente (`pagina-web-servicios`).
2. Guarda las imágenes en `public/proyectos/<slug>/` (por ejemplo `Portada.png`, `Imagen-1.png`). El nombre de la carpeta debe coincidir con el `id`.
3. Impórtalo y añádelo al array de `data/proyectos/index.ts`. El orden del array es el orden en la web.

7.3.1. Markdown admitido en readme

El renderizador propio de `components/proyectos.tsx` admite:

- Encabezados `## Título` (cada uno crea una pestaña) y `### Subtítulo`.
- Listas con `-` y numeradas con `1. .`
- Negrita `**texto**` y código en línea con comillas invertidas.
- Bloques de código con tres comillas invertidas.
- Tablas con `|`.

No admite enlaces, imágenes, cursiva ni listas anidadas. Dentro de una cadena de plantilla de TypeScript, las comillas invertidas del Markdown deben escaparse con una barra invertida (`\``).

8 Componentes

8.1 Página principal (app/page.tsx)

Monta, en orden: SiteHeader, Hero, About, Services, Process, Proyectos, Contact, SiteFooter y BackToTop. Las secciones, salvo Hero, se envuelven en Reveal para la animación de entrada. **Para reordenar o quitar una sección, edita este archivo.**

8.2 Cabecera (site-header.tsx)

- Menú fijo con enlaces definidos en el array ENLACES (cada href es el id de una sección).
- Resalta la sección visible usando el hook useActiveSection.
- En móvil muestra siempre el botón de contacto y un menú desplegable.
- El enlace “Enviar correo” abre MailModal.

Para añadir un enlace al menú, añade una entrada a ENLACES y asegúrate de que la sección tiene ese id.

8.3 Hero (hero.tsx)

Contiene el avatar (public/images/perfil.jpg), el nombre, el cargo, la descripción, dos botones (“Solicitar colaboración gratuita” y “Enviar correo”) y tres ventajas. Los textos están escritos directamente en el archivo.

8.4 Sobre mí (about.tsx)

Los arrays highlights (cuatro destacados) y facts (ubicación y coste) se editan al inicio del archivo. Los párrafos de presentación están en el JSX.

8.5 Servicios (services.tsx)

Muestra tarjetas desplegables: en escritorio se abren al pasar el ratón y en móvil al tocar (estado openIndex). Debajo muestra la lista de tecnologías.

8.6 Proceso (process.tsx)

Array pasos con tres pasos y un bloque “¿Por qué es gratis?”. Edita los textos directamente.

8.7 Proyectos (proyectos.tsx)

Es el componente más grande. Incluye:

- Cuadrícula de tarjetas con la portada de cada proyecto.
- ProyectoModal: ventana con galería, título, etiquetas, pestañas de secciones y enlace a GitHub.

- **Galeria:** carrusel con miniaturas, flechas, teclado y gesto táctil de deslizar. Altura fija para que las flechas no se muevan.
- **Visor:** pantalla ampliada con zoom (1x a 6x) mediante rueda, barra, teclas +, -, 0 y pellizco con dos dedos; arrastre para mover.
- **renderMarkdown:** renderizador Markdown mínimo.

8.8 Contacto (`contact.tsx`)

Columna izquierda con detalles (zona, tiempo de respuesta, modalidad) en el array `details`; columna derecha con un aviso de gratuidad y el formulario.

8.9 Pie de página (`site-footer.tsx`)

Tres columnas: identidad, información legal y perfil profesional (LinkedIn). Contiene dos modales:

- **Legal:** aviso legal, privacidad (RGPD), términos de gratuidad y cookies. **El texto legal está escrito aquí;** si cambias las condiciones del servicio, actualízalo también en `contact-form.tsx`.
- **Documentación PDF:** muestra `public/Documentacion.pdf` en un `iframe`. En móviles, donde los navegadores no incrustan PDFs, ofrece abrirlo o descargarlo.

8.10 Utilidades de interfaz

`reveal.tsx` Anima la entrada de cada sección. Si JavaScript falla o el usuario prefiere “reducir movimiento”, el contenido es visible desde el principio.

`back-to-top.tsx` Botón que aparece tras 700 px de scroll.

`mail-modal.tsx` Confirma antes de abrir el cliente de correo con un asunto mínimo y un ID único.

`ui/button.tsx` Define `buttonVariants` (variantes y tamaños) usado en enlaces y botones.

8.11 Hooks y utilidades (`lib/`)

`useScrollLock(activo)` Bloquea el scroll del fondo mientras un modal está abierto. Usa un contador compartido para que varios modales simultáneos no se pisen.

`useActiveSection(ids)` Devuelve el id de la sección que ocupa la franja central de la pantalla.

`cn(...)` Combina clases de Tailwind resolviendo conflictos (`clsx` + `tailwind-merge`).

9 Formulario de contacto

9.1 Flujo completo

1. El visitante rellena el formulario. Cada campo se valida al perder el foco.

2. Debe aceptar las condiciones. Si no las ha leído hasta el final, se abre el modal de términos y la casilla solo se habilita al llegar al final del texto (se recuerda durante la sesión con `sessionStorage`).
3. Al pulsar “Preparar mensaje” se abre un modal con dos opciones.
4. **Enviar desde mi aplicación de correo:** abre un enlace `mailto:` con asunto y cuerpo rellenos.
5. **Copiar el mensaje manualmente:** muestra tres bloques (Para, Asunto, Mensaje) con botón de copiar.
6. Una pantalla final recuerda que el correo *no se envía solo* y permite limpiar el formulario.

El ID de registro y la fecha de aceptación se generan en el instante exacto del envío o copia, no al abrir el formulario.

9.2 Validaciones

Campo	Regla
Nombre	Obligatorio, de 2 a 100 caracteres.
Correo	Obligatorio, expresión regular básica de correo.
Teléfono	Opcional; si se rellena, se valida con la regla del país elegido.
Servicio	Obligatorio (lista <code>serviceOptions</code>).
Asunto	Obligatorio, de 3 a 150 caracteres.
Mensaje	Obligatorio, de 10 a 2000 caracteres.
Términos	Casilla obligatoria.

Las funciones `validarNombre`, `validarEmail`, etc., están al inicio de `contact-form.tsx`. Para cambiar un límite, edita la función correspondiente y el atributo `maxLength` del campo.

9.3 Identificador único (`lib/mail.ts`)

```
ID-YYYYMMDD-HHMMSS-MSHEX-TZ
ejemplo: ID-20261005-190143-427A3F-UTC+0200
```

Compone fecha y hora locales, milisegundos, cuatro dígitos hexadecimales aleatorios y la zona horaria. Se incluye en el asunto y el cuerpo del correo para dar trazabilidad.

9.4 Países y teléfonos (`lib/paises.ts`)

Una tabla compacta en texto (`ISO prefijo Nombre`) se convierte en la lista `PAISES`. Los países más habituales (`DESTACADOS`) aparecen primero. Las reglas de validación específicas están en `ESPECIFICOS`; los demás usan una regla genérica de 4 a 14 dígitos. Para añadir una regla propia de un país, añade su código ISO a `ESPECIFICOS` con una expresión regular y un ejemplo.

9.5 Cómo cambiar el destino del correo

No se escribe en el código. Se define en GitHub como variable de repositorio `CONTACT_EMAIL` (ver sección de despliegue). En local, mediante `.env.local`.

Aviso de privacidad realista: al ser un sitio estático, la dirección final sí queda en los archivos publicados (el navegador la necesita para abrir el correo), pero no en el código fuente del repositorio.

9.6 Si quieres un envío real sin cliente de correo

El diseño actual evita servicios de terceros. Si en el futuro se desea enviar sin que el visitante use su correo, sería necesario integrar un servicio externo (por ejemplo, un formulario alojado o una función serverless) y modificar `handleEnviarCorreo` en `contact-form.tsx`. Eso cambiaría también las condiciones legales de privacidad del sitio.

10 Estilos y tema

10.1 Tokens de color (app/globals.css)

Los colores se definen como variables CSS en el espacio OKLCH dentro de `:root` y se exponen a Tailwind mediante `@theme inline`.

Variable	Uso
<code>-background</code>	/ Fondo y texto general.
<code>-foreground</code>	
<code>-primary</code>	Color de acento (azul). Botones, enlaces, iconos destacados.
<code>-secondary</code> , <code>-muted</code> , <code>-accent</code>	Fondos suaves de secciones y chips.
<code>-border</code> , <code>-input</code> , <code>-ring</code>	Bordes, campos y anillo de foco.
<code>-ink</code>	/ Fondo oscuro del <i>hero</i> , cabecera, pie y 404, y su texto.
<code>-ink-foreground</code>	
<code>-destructive</code>	Errores.
<code>-radius</code>	Radio base de bordes redondeados.

Para **cambiar la paleta**, modifica los valores de `:root`. Cambiar solo `-primary` recolora todos los acentos del sitio. Si cambias `-ink`, actualiza también `themeColor` en `app/layout.tsx` (hexadecimal equivalente) para que el color de la barra del navegador coincida.

10.2 Otros estilos globales

- `scroll-padding-top: 4.5rem`: evita que la cabecera fija tape los títulos al saltar a un ancla.
- Barra de desplazamiento discreta, sin flechas, con alternativa para Firefox.
- Estilos `[data-reveal]` para la animación de entrada, con reglas de `prefers-reduced-motion`.

10.3 Fuente

Se carga *Inter* desde `next/font/google` en `layout.tsx` y se asigna a la variable `-font-inter`. Para cambiarla, sustituye el `import` y el nombre de la fuente.

10.4 Modo claro

El sitio declara `color-scheme: light` y no implementa modo oscuro. Existen variantes `dark:` en el botón de shadcn, pero no hay tokens oscuros definidos.

11 SEO y metadatos

11.1 `app/layout.tsx`

- **metadata:** título, descripción, palabras clave, URL canónica, robots, iconos, Open Graph y Twitter Card.
- **viewport:** esquema de color y color de tema.
- **jsonLd:** grafo Schema.org con una `Person` y un `ProfessionalService` cuyo catálogo de ofertas se genera desde `data/servicios.ts`.

El JSON-LD se serializa escapando el carácter `<` para impedir que un valor cierre accidentalmente la etiqueta `<script>`.

Si cambias nombre, ciudad o descripción, debes actualizarlos en varios sitios de `layout.tsx`: `metadata`, `openGraph`, `twitter` y `jsonLd`.

11.2 Imagen para redes (`public/og\_image.png`)

Debe medir 1200×630 píxeles. Es la imagen que se muestra al compartir el enlace.

11.3 `sitemap.ts` y `robots.ts`

Ambos se generan en el build con la URL real y están marcados como `force-static`. El `sitemap` incluye la página principal y las imágenes de perfil y proyectos. Los buscadores solo leen `/robots.txt` en la raíz del dominio; esto funciona porque el repositorio se llama `<usuario>.github.io`.

11.4 Icono

`public/icon.svg` se usa como favicon e icono de Apple. Es un SVG con un símbolo de terminal.

12 Despliegue en GitHub Pages

12.1 Flujo automático

El archivo `.github/workflows/deploy.yml` se ejecuta al hacer *push* a `main` o manualmente (`workflow\_dispatch`). Tiene dos trabajos:

1. **build:** descarga el código, instala pnpm y Node 24, instala dependencias con `-frozen-lockfile`, configura Pages, restaura la caché de Next.js, ejecuta `pnpm build` y sube `./out` como artefacto.
2. **deploy:** publica el artefacto en el entorno `github-pages`.

12.2 Configuración inicial del repositorio

1. En GitHub: **Settings** → **Pages** → **Source: GitHub Actions**.
2. En **Settings** → **Secrets and variables** → **Actions** → **Variables**, crea la variable `CONTACT\_EMAIL` con la dirección de contacto.
3. Haz *push* a `main`. El workflow publica el sitio.

12.3 Nombre del repositorio y basePath

- Si el repositorio se llama `<usuario>.github.io`, el sitio se sirve en la raíz y el `basePath` queda vacío.
- Si es un repositorio de proyecto (por ejemplo `inicio`), se sirve en `/inicio/` y el workflow inyecta ese prefijo automáticamente mediante `actions/configure-pages`.
- Si cambias el nombre del repositorio o usas un dominio propio, ajusta `NEXT\_PUBLIC\_SITE\_ORIGIN` (o el valor por defecto en `site-config.ts`).

12.4 El archivo .nojekyll

Está en `public/` y evita que GitHub Pages procese el sitio con Jekyll, que ignoraría las carpetas que empiezan por guion bajo, como `\_next`.

12.5 Caché de compilación

El workflow guarda `.next/cache` con una clave basada en el hash de `pnpm-lock.yaml` y de los directorios de código. Acelera builds sin cambios de dependencias.

13 Recetas: cómo modificar...

13.1 Cambiar mi nombre o cargo

Busca el texto en: `app/layout.tsx` (metadatos y JSON-LD), `components/hero.tsx`, `components/site-footer` y `README.md`. Actualiza también el texto legal del pie.

13.2 Cambiar la foto de perfil

Sustituye `public/images/perfil.jpg` manteniendo el nombre, o cambia la ruta en `hero.tsx` y en el JSON-LD de `layout.tsx`.

13.3 Cambiar la ciudad

Actualízala en `layout.tsx` (descripción, `addressLocality`, `areaServed`), `about.tsx` (array `facts`), `contact.tsx` y `site-footer.tsx`.

13.4 Cambiar textos de una sección

Abre el componente de la sección (tabla de la sección *Componentes*) y edita el texto dentro del JSX o de los arrays del inicio.

13.5 Cambiar el color principal

Edita `-primary` y `-ring` en `app/globals.css`. Se aplica a todo el sitio.

13.6 Ocultar la insignia de disponibilidad

Pon `disponible = false` en `lib/site-config.ts`.

13.7 Reordenar o eliminar una sección

Edita `app/page.tsx`. Si eliminas una sección, quita también su enlace de ENLACES en `site-header.tsx`.

13.8 Añadir una sección nueva

1. Crea `components/mi-seccion.tsx` con un `<section id="mi-seccion">`.
2. Impórtala en `app/page.tsx` y colócala (con `Reveal` si quieres animación).
3. Añade `{ href: 'mi-seccion', label: 'Mi secci'on' }` a ENLACES en `site-header.tsx`.

13.9 Sustituir el PDF de documentación

Reemplaza `public/Documentacion.pdf` conservando el nombre, o cambia la constante `DOCS\_PDF\_PATH` en `site-footer.tsx`.

13.10 Cambiar el texto legal o las condiciones

Hay dos copias que deben mantenerse coherentes:

- Modal de términos y cuerpo del correo en `components/contact-form.tsx` (función `generarContenido` y el modal).
- Aviso legal en `components/site-footer.tsx`.

Consulta con un profesional del derecho antes de modificar el contenido legal.

13.11 Añadir un país o cambiar el país por defecto

El país por defecto es el primero de `PAISES` (España). Para cambiarlo, reordena `DESTACADOS` en `lib/paises.ts`.

13.12 Cambiar el prefijo del asunto del correo

En `contact-form.tsx`, la variable `asuntoCorreo` comienza por `[inicio-github.io]`. En `lib/mail.ts`, `generarMailtoUrl` usa “Contacto desde ...”.

13.13 Cambiar el tiempo de respuesta

Aparece en `hero.tsx`, `contact.tsx`, `process.tsx` y en el mensaje final de `contact-form.tsx`. Busca “24 horas”.

13.14 Reutilizar este sitio como plantilla

1. Haz una copia del repositorio con el nombre `<tu-usuario>.github.io`.
2. Cambia `siteOrigin` en `site-config.ts`.
3. Sustituye foto, imágenes, PDF y `og\_image.png`.
4. Edita los datos de `data/` y los textos de los componentes.
5. Define la variable `CONTACT\_EMAIL` en tu repositorio.
6. Actualiza el texto legal con tus datos.

Atención: el README indica “todos los derechos reservados”. Contacta con el autor antes de reutilizar el código.

14 Accesibilidad

- Los modales usan `role="dialog"`, `aria-modal="true"` y `aria-labelledby`.
- Se cierran con **Escape**, el botón de cerrar o haciendo clic fuera. El scroll del fondo se bloquea y se restaura.
- Los iconos decorativos llevan `aria-hidden="true"` y los botones solo de icono, `aria-label`.
- Los errores de formulario usan `role="alert"` y `aria-invalid`.
- El menú marca la sección actual con `aria-current`.
- Las animaciones se desactivan con “reducir movimiento”.

Al añadir componentes nuevos, mantén estas prácticas.

15 Resolución de problemas

Síntoma	Causa probable y solución
El formulario muestra “falta configurar el correo”	<code>NEXT_PUBLIC_CONTACT_EMAIL</code> está vacía. En local, crea <code>.env.local</code> ; en GitHub, define la variable <code>CONTACT_EMAIL</code> .
Las imágenes no se ven en producción	Falta <code>asset()</code> en la ruta, o el archivo no está en <code>public/</code> .
El build falla en GitHub	Error de TypeScript (se trata como fallo) o <code>pnpm-lock.yaml</code> desactualizado. Ejecuta <code>pnpm build</code> en local.
El sitio se ve sin estilos o da 404 en <code>_next</code>	Falta <code>public/.nojekyll</code> o el <code>basePath</code> no coincide con el nombre del repositorio.
No se abre el cliente de correo	El visitante no tiene aplicación de correo asociada a <code>mailto:..</code> . Usa la opción “Copiar el mensaje”.

Síntoma	Causa probable y solución
El PDF no aparece en el móvil	Comportamiento esperado; se ofrece abrir o descargar.
Un servicio nuevo sale con icono de llave	Falta añadir su <code>id</code> al objeto <code>ICONOS</code> de <code>services.tsx</code> .
Un proyecto no aparece	No se añadió al array de <code>data/proyectos/index.ts</code> .
Las imágenes de un proyecto no cargan	La carpeta de <code>public/proyectos/</code> no coincide con el <code>id</code> , o las rutas del <code>index.ts</code> tienen errores de mayúsculas.

16 Buenas prácticas al contribuir

- Ejecuta `pnpm build` antes de hacer *push*: es lo mismo que hará el servidor.
- No escribas datos personales (correo, teléfono) en el código: el repositorio es público.
- Usa siempre `asset()` para rutas de `public/`.
- Mantén los textos en español y la coherencia entre el texto legal y el cuerpo del correo.
- Prefiere las clases de Tailwind y los tokens de color en lugar de colores fijos.
- Haz commits pequeños con mensajes descriptivos.

17 Autor y licencia

Redwan Alexandru Teodor – Técnico en Sistemas y Redes, Sevilla, España.

LinkedIn: <https://www.linkedin.com/in/redwan-alexandru-teodor/>

© 2026 Redwan Alexandru Teodor. Todos los derechos reservados. El código se publica como referencia del trabajo del autor; para reutilizarlo, adaptarlo o usarlo en otro proyecto, es necesario contactar previamente con él.